

INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO



Materia: Administración y organización de datos

Trabajo por desarrollar: Reporte Portafolio

Maximiliano Mendoza Vizcarra R24170107

Semestre: (Enero-Junio 2026)

Especialidad: Ing. Informática

Nombre del Docente: Ing. Jesus Salas

1. Introducción Ejecutiva

¿De qué trata este portafolio?

A lo largo del semestre desarrollé cuatro proyectos para la materia de Administración y Organización de Datos. Cada uno atacó un problema distinto, pero todos tenían algo en común: nada de bases de datos. Eso obligó a pensar bien qué formato usar para guardar la información y por qué, algo que normalmente se omite cuando ya tienes un motor de BD haciendo todo el trabajo pesado por detrás.

El recorrido fue de menos a más: empezamos con texto plano para guardar lecturas de sensores, luego usamos JSON para configurar un dispositivo IoT sin tocar el código, después analizamos datos de ventas con CSV y Pandas, y al final medimos en números reales cuánto tarda y cuánto pesa cada formato cuando tienes veinte millones de registros. Ese fue el hilo conductor del semestre.

Resumen de proyectos

Proyecto	Tecnologías	Formatos
Corte 1 — Data Bridge (Invernadero inteligente)	Python, PHP, HTML/CSS	.txt secuencial
Corte 2 — Control de Acceso (Monitor IoT Pico W)	MicroPython, PHP, Chart.js	.json (config) y .txt (log)
Corte 3 — Actividad Integradora (Ventas)	Python, Pandas, Matplotlib	.csv y .png
Corte 4 — Evaluación General (Hospital)	Python, Pandas, ijson, Faker	.csv, .json y .txt

2. Contenido Técnico de los Proyectos

Corte 1 — Data Bridge (Invernadero inteligente)

El primer sistema simula un invernadero con sensores de temperatura, humedad y nivel de agua. Python genera los registros y los guarda en un archivo de texto. Desde una página en PHP el usuario escribe un criterio de búsqueda, eso se pasa a un script de Python que filtra el archivo maestro y guarda los resultados, y luego PHP los muestra en una tabla. También tiene un endpoint que recibe datos de sensores físicos reales por GET.

Usé texto plano porque los datos son una línea por registro, nada más. Meter JSON o CSV aquí hubiera sido innecesario. Lo único importante era que el modo append en receptor.php me dejara agregar registros sin borrar los anteriores, y eso lo hace cualquier archivo de texto.

Cómo fluyen los datos

Paso	Qué pasa	Operación
1	generador.py crea los 1 000 registros de sensores	write() → maestro.txt
2	El usuario escribe qué buscar y procesar.php lo guarda	write() → busqueda.txt
3	persistencia.py lee maestro.txt y saca solo lo que coincide	read + write() → filtrado.txt
4	tabla.php abre filtrado.txt y lo muestra como tabla HTML	read() → pantalla
5	receptor.php recibe datos reales de sensores por GET	append() → maestro.txt

Por qué se eligió cada formato

Archivo	Formato	Justificación
maestro.txt	TXT	Los sensores escriben constantemente. TXT secuencial con append es lo más rápido: no hay que leer nada antes de escribir, solo se agrega al final.
busqueda.txt	TXT	Solo guarda una palabra o número: el criterio de búsqueda. Usar JSON para eso hubiera sido exagerado.
filtrado.txt	TXT	Guarda el último resultado para que PHP no tenga que pedirle a Python que reprocese el archivo maestro en cada recarga de página.

Corte 2 — Control de Acceso / Monitor IoT (Raspberry Pi Pico W)

Para el segundo corte conecté un sensor de temperatura DS18B20 a una Raspberry Pi Pico W. Cuando la temperatura supera el umbral configurado, el sistema activa un relé que enciende un ventilador de 12V. Los datos se mandan por WiFi a un servidor PHP que los guarda y los muestra en un dashboard con una gráfica y un velocímetro animado parecido a LabVIEW.

Aquí apareció JSON por primera vez, pero solo para la configuración. La idea era poder cambiar el SSID, la contraseña WiFi, el umbral de temperatura y los pines GPIO sin tener que modificar el código del firmware. Eso es especialmente útil cuando estás trabajando con hardware y no quieres estar resubiendo código cada vez que cambias algo de configuración.

Cómo fluyen los datos

Paso	Qué pasa	Operación
1	main.py lee Config.json al arrancar (WiFi, GPIO, umbral)	json.load() — lectura única
2	El sensor mide temperatura y la Pico la manda al servidor	HTTP GET con datos JSON
3	Guardar_Datos.php recibe el JSON y lo agrega al log	FILE_APPEND → auditoria.txt
4	Lecturas.php devuelve las últimas 20 lecturas	read() → respuesta JSON
5	Index.php muestra gauge + gráfica actualizada por AJAX	Consulta cada pocos segundos

Por qué se eligió cada formato

Archivo	Formato	Justificación
Config.json	JSON	JSON permite organizar los parámetros con claves claras, incluso con bloques anidados como el de pines GPIO. Se lee una sola vez al arrancar, así que el costo de parseo no importa. Lo que importa es poder modificarlo sin tocar el firmware.
auditoria.txt	TXT (log)	El sensor escribe cada segundo. FILE_APPEND garantiza que cada escritura sea rápida y no borre el historial. No necesitamos leer el archivo para poder escribir en él.

Corte 3 — Actividad Integradora (Análisis de ventas)

Este proyecto analiza ventas de una tienda de tecnología. Hay un CSV con 18 registros (tres productos en seis meses) y el script calcula totales, promedios y tendencias con Pandas, luego genera tres gráficas distintas: barras, líneas y pastel. Todo se imprime en consola y las gráficas se guardan en PNG.

Fue el proyecto donde se usó CSV como formato principal y se notó de inmediato por qué es tan popular para análisis: `pd.read_csv()` lo carga directo a un DataFrame, y desde ahí `groupby`, `sum` y `mean` funcionan sin parsear nada a mano. Si hubiera usado TXT con ese mismo dataset, habría tenido que escribir el parser yo mismo.

Cómo fluyen los datos

Paso	Qué pasa	Operación
1	ventas_tecnologia.csv — 18 registros tabulares de entrada	<code>pd.read_csv()</code> completo
2	analisis_ventas.py calcula agrupaciones y estadísticas con Pandas	Operaciones en DataFrame (RAM)
3	Matplotlib genera 3 gráficas y las guarda en un PNG	<code>plt.savefig(dpi=150)</code> → .png

Por qué se eligió cada formato

Archivo	Formato	Justificación
ventas_tecnologia.csv	CSV	Los datos de ventas son completamente tabulares: una fila por registro, columnas fijas. CSV es el formato natural para eso y tiene soporte directo en Pandas sin configuración extra.
graficas_ventas.png	PNG	Sin pérdida de calidad. Con <code>dpi=150</code> quedan bien para incluirlas en reportes o presentaciones.

Corte 4 — Evaluación General (Sistema hospitalario)

El último proyecto fue el más completo. Se generan registros de pacientes con datos ficticios en español mexicano usando Faker, y se guardan en CSV y en JSON al mismo tiempo. Hay un menú interactivo para buscar pacientes, agregar registros y generar reportes. También hay un módulo de análisis estadístico con Pandas, y lo más importante del semestre para mí, un sistema de benchmarking que mide en segundos cuánto tarda cada formato en escribir y leer distintas cantidades de datos.

La parte más complicada fue leer archivos JSON de varios gigabytes. Si intentas cargar un JSON de 4 GB con `json.load()` normal en una máquina con 4 GB de RAM, el programa se cae. La solución fue usar `ijson`, que lee el archivo como un stream: en vez de cargarlo todo de golpe, lo recorre registro por registro. Eso funcionó incluso a 20 millones de registros.

Cómo fluyen los datos

Módulo	Qué hace	Operación
Generación	generar_datos.py crea pacientes con Faker (es_MX)	write() → .csv y .json
CRUD	main.py + sistema.py — menú para buscar y agregar	DictReader / ijson + append
Análisis	analisis.py — estadísticas descriptivas con Pandas	pd.read_csv() → reportes .csv y .txt
Benchmarking	experimentacion.py — mide tiempos y tamaños en disco	time.time() → resultados.csv
Gráficas	visualizacion.py — 5 gráficas de rendimiento	plt.savefig() → graficas/*.png

Por qué se eligió cada formato

Archivo	Formato	Justificación
pacientes_N.csv	CSV	Para datos tabulares de pacientes, CSV fue más rápido y ocupó mucho menos espacio. A 1 millón de registros: 75.7 MB en CSV contra 198.73 MB en JSON.
pacientes_N.json	JSON + ijson	JSON permite anidar los datos de la consulta médica dentro de cada paciente. Se usa ijson para leerlo sin cargar el archivo completo en RAM.
resultados.csv	CSV (append)	Acumula resultados de cada corrida de benchmarking. Con append se pueden hacer varias pruebas en distintos momentos sin perder lo anterior.
reportes .txt	TXT	Reportes de texto generados con f-strings. Modo 'w' para que siempre reflejen el análisis más reciente.

3. Evaluación General del Sistema

3.1 Resultados del benchmarking

Las pruebas escalaron desde 1 000 hasta 20 millones de registros midiendo tres cosas: tiempo de escritura, tiempo de lectura y cuánto pesa el archivo en disco. Se corrió para CSV y para JSON. El resultado más concreto: a un millón de registros, CSV escribe en 15 segundos y JSON tarda casi 30. Y el JSON pesa más del doble.

Registros	Formato	Escritura (s)	Lectura (s)	Tamaño (MB)
1 000	CSV	0.02	0.01	0.07
1 000	JSON	0.03	0.01	0.20
1 000 000	CSV	15.36	1.91	75.7
1 000 000	JSON	29.85	3.19	198.73
10 000 000	CSV	156.52	23.84	766.59
10 000 000	JSON	295.68	33.34	1 996.82
20 000 000	CSV	305.99	37.17	1 543.72
20 000 000	JSON	624.33	67.23	4 004.19

A 20 millones de registros el JSON ya pesa 4 GB. Intentar cargarlo con `json.load()` normal en una laptop de 4 o 8 GB de RAM simplemente falla. Por eso fue necesario `ijson`. El CSV en la misma escala llega a 1.5 GB y se lee en 37 segundos, lo cual ya es lento pero al menos funciona sin trucos adicionales. Si esto fuera un hospital real con datos de todo un estado, CSV seguiría siendo viable sin infraestructura extra; JSON ya no.

3.2 Análisis costo-beneficio

Cada vez que elegí un formato o una forma de abrir un archivo hubo un tradeoff: algo que costaba a cambio de algo que se ganaba. Esto es un resumen de los más importantes:

Decisión	Lo que cuesta	Lo que se gana
filtrado.txt como caché (Corte 1)	Una escritura extra cada vez que se filtra	PHP no tiene que pedirle a Python que reprocese el archivo maestro en cada petición
JSON para Config.json (Corte 2)	Unos milisegundos de parseo al arrancar	Cambiar parámetros sin recompilar ni subir firmware nuevo al microcontrolador
Append en logs (Cortes 1 y 2)	El archivo crece indefinidamente	Escritura rápida sin tener que leer lo anterior; aguanta escrituras cada segundo
CSV sobre JSON para datos masivos (Corte 4)	Sin estructura jerárquica ni datos anidados	Casi el doble de velocidad y 61% menos espacio en disco a 1M registros
ijson en lugar de <code>json.load()</code> completo	Código más difícil de escribir y de entender	Permite leer archivos de 4 GB en equipos con poca RAM sin crashear

4. Conclusiones y Enlace al Repositorio

¿Qué aprendí sobre guardar datos antes de llegar a las bases de datos relacionales?

Antes de empezar el semestre, si alguien me decía "guarda esos datos" yo pensaba automáticamente en una base de datos con tablas, llaves primarias y queries. Esta materia me obligó a construir esos sistemas sin esa herramienta, y eso cambió bastante cómo entiendo el problema.

Lo que más me quedó claro es que la decisión del formato no es trivial. Parece algo secundario pero tiene consecuencias reales: en el Corte 4 la diferencia entre CSV y JSON a 20 millones de registros fue la diferencia entre un archivo que entra en disco fácil y otro que ya pesa 4 GB. Eso en un proyecto real significa costos de infraestructura distintos.

También entendí que los conceptos de los motores de base de datos no aparecen de la nada: el modo append que usé en los logs es básicamente lo mismo que el write-ahead log de PostgreSQL. El archivo filtrado.txt que actúa como caché es el principio de un índice. El streaming con ijson es lo que hacen los cursores de BD para no cargar millones de filas en RAM de una vez. Verlo en archivos planos antes de verlo en un motor hace que tenga mucho más sentido.

Comparativa de los tres formatos trabajados en el semestre

Criterio	TXT plano	JSON	CSV
Velocidad de escritura	Muy alta	Media	Alta
Velocidad de lectura	Alta	Media	Muy alta
Espacio que ocupa	Mínimo	Alto	Bajo
Estructura soportada	Plana	Jerárquica	Tabular
Integración con Pandas	Manual	Indirecta	Nativa
Cuándo usarlo	Logs e IoT	Config y datos anidados	Análisis y reportes

Enlace del Repositorio Inicial del proyecto

<https://github.com/r24170107-commits/Organizacion-Archivos-Portfolio>

Enlaces de los Repositorios Del Portafolio De Arquitectura y Organizacion De Datos

Proyectos	Tecnologias	Formatos de Archivo	Enlace Directo
Proyecto-Corte-1-Data-bridge	Python, PHP, HTML	.txt	https://github.com/r24170107-commits/Organizacion-Archivos-Portfolio/tree/main/Corte_1-Data-bridge
Proyecto-Corte-2-Control de acceso	MicroPython, PHP, HTML/CSS/JS	.json, .txt	https://github.com/r24170107-commits/Organizacion-Archivos-Portfolio/tree/main/Corte_2-Control de acceso
Proyecto-Corte-3-Actividad-integradora	Python, Pandas, Matplotlib	.csv, .png	https://github.com/r24170107-commits/Organizacion-Archivos-Portfolio/tree/main/Corte_3-Actividad-Integradora
Proyecto-Corte-4-Evaluacion_General_OA	Python, Pandas, Matplotlib, ijson, Faker	.csv, .json, .txt	https://github.com/r24170107-commits/Organizacion-Archivos-Portfolio/tree/main/Corte_4-Evaluacion_General_OA